

A method for numerically approximating ordinary differential equations

Hadrian Ward
MA2051

A long time ago I was working on a sailboat simulator game in Python which involved calculating forces and updating a physics simulation. All the forces (on the sails and on the boat's hull) involved the drag equation: $f = \frac{1}{2}AC_dV^2r$ where A is the area exposed to the fluid flow, C_d is a constant based on the shape, V is the velocity, and r is the fluid density (NASA). Basically everything is a constant multiplied by V^2 . Every time step of the simulation, the boat's position was updated by the velocity and the velocity was updated by the acceleration (net force divided by mass), basically modelling the equations: $P(t) = \int V(t)dt$, $V(t) = \int f_{ext}(t) - C_dV^2dt$. All constants from the drag equation have been condensed to C_d , and mass is assumed to be 1 for simplicity. f_{ext} is added to illustrate effects later on. Everything from now only concerns the velocity and acceleration, not position.

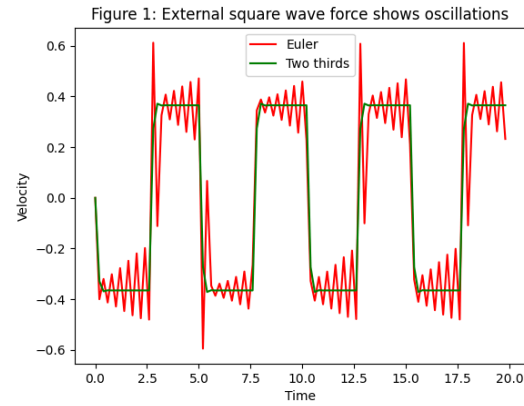
In reality the program used to update the variables in discrete time steps (also known as Euler Integration): $\Delta P = V\Delta t$, $\Delta V = (-C_dV^2 + f_{ext}(t))\Delta t$

These equations were straightforward to program into the physics simulation. However there was an issue with the simulation's stability. Drag is a form of negative feedback which leads to falling objects (or drifting boats) moving at a constant speed as time approaches infinity. Euler integration with sufficiently small step sizes will work fine.

When the step sizes are large, drag is large, or external forces vary quickly, then euler integration runs into stability problems: If ΔV brings the velocity past equilibrium velocity

(terminal velocity in the case of gravity or the ideal physical value of the velocity if the external force is changing) then it “overshoots” and goes back the other way on the next time step. Even worse, if the ΔV is more than twice the difference between V and the equilibrium then its oscillation will grow exponentially until the computer runs into an overflow error. Early versions of my sailboat simulator had this exact problem which made it unusable.

The solution I came up with in the summer of 2021 was to step the simulation by $\frac{2}{3}\Delta t$, save the state, step by another $\frac{2}{3}\Delta t$ (total of $\frac{4}{3}\Delta t$ at this point), then average the two states which is an average time step of $1\Delta t$. This integration method fixed the problem and I haven’t had problems with it since. It eliminates oscillation by cancelling out successive opposite changes and only steps by the average change. Therefore it is extremely stable and useful when the time steps or accelerations are large and where euler integration might oscillate.



To more concretely prove this, first we need to derive the terminal velocity with a given constant external force and $C_d: V_{eq}^2 C_d = f_{ext}, V_{eq} = \sqrt{\frac{f_{ext}}{C_d}}$. Next, linearize dV near V_{eq} : The derivative of $\frac{dV}{dt}$ with respect to V is: $\frac{dV}{dt} (-C_d V^2 + f_{ext}(t)) = -2C_d V$. Therefore with V close to V_{eq} , $\frac{dV}{dt} = -2C_d V(V - V_{eq})$ or with discrete steps: $\Delta V = -2C_d V(V - V_{eq})\Delta t$. Euler integration will oscillate if $-\Delta V > V - V_{eq}$ so $2C_d V(V - V_{eq})\Delta t > V - V_{eq}$ or $2C_d V\Delta t > 1$. Therefore euler integration will not oscillate when the following condition holds: $2C_d V\Delta t < 1$.

It is also slightly more accurate than Euler integration. Updating a function with Euler integration goes as follows: $f(t + \Delta t) = f(t) + f'(t)\Delta t$ whereas the two-thirds technique is:

$$f(t + \Delta t) = \frac{f(t) + \frac{2}{3}f'(t)\Delta t + f(t) + \frac{2}{3}f'(t)\Delta t + \frac{2}{3}f'(t + \frac{2}{3}\Delta t)\Delta t}{2} = \frac{2f(t) + \frac{4}{3}f'(t)\Delta t + \frac{2}{3}f'(t + \frac{2}{3}\Delta t)\Delta t}{2}$$

$$= f(t) + \frac{2}{3}f'(t)\Delta t + \frac{1}{3}f'(t + \frac{2}{3}\Delta t)\Delta t$$

As Δt approaches 0, the method converges

on the correct answer, although the multiple steps make it more accurate than Euler.

Figure 2 shows the result of a simulation with

$V(0) = 2$, $C_d = 15$, and no external force.

Solving the differential equation for the ideal velocity:

$$\frac{dV}{dt} = -15V^2$$

$$-\frac{1}{15} \int \frac{dV}{V^2} = \int dt, \frac{1}{15}V^{-1} = t + C, V = \frac{1}{15(t+C)}$$

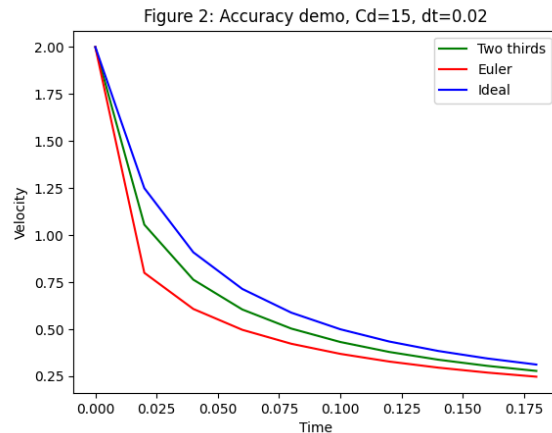
Solving for C:

$$V(0) = 2 = \frac{1}{15C}, C = \frac{1}{30}$$

$$V = \frac{1}{15(t + \frac{1}{30})}$$

Figure 2 compares this ideal function with both the two-thirds and Euler methods.

I found out while writing this that what I came up with is very similar to Heun's method which has the update rule: $f(t + \Delta t) = f(t) + \frac{\Delta t}{2}(f'(t) + f'(t + \Delta t))$.



References:

Drag equation, NASA:

<https://www.grc.nasa.gov/WWW/k-12/VirtualAero/BottleRocket/airplane/drageq.html>

Heun's method, Brown University:

<https://www.cfm.brown.edu/people/dobrush/am33/Mathematica/ch3/heun.html>